

POLITECNICO
MILANO 1863

Best Bike Paths, Luca Suzani, Alberto Spezzapria, Alessio Pesavento

ITD Document

Deliverable:	ITD
Title:	ITD Document
Authors:	Luca Suzani, Alberto Spezzapria, Alessio Pesavento
Version:	1.0
Date:	01-February-2026
Download page:	https://github.com/Alessio01103/PesaventoSpezzapriaSuzani.git
Copyright:	Copyright © 2026, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Contents

Table of Contents . . . 3
List of Figures . . . 5
List of Tables . . . 5

1 Introduction **7**
1.1 Purpose . . . 7
1.2 Scope . . . 7
1.3 Definitions, Acronyms, and Abbreviations . . . 8
1.3.1 Definitions . . . 8
1.3.2 Acronyms . . . 8
1.3.3 Abbreviations . . . 9
1.4 Revision History . . . 9
1.5 Reference Documents . . . 9
1.6 Document Structure . . . 9
1.7 Use of Generative AI Tools . . . 10

2 Implemented Requirements and Functions **12**
2.1 Implemented Requirements . . . 12
2.1.1 Implemented Functional Requirements . . . 12
2.1.2 Excluded or Partially Implemented Functionalities . . . 12
2.1.3 Additional Functionalities . . . 13

3 Adopted Development Frameworks and Technologies **14**
3.1 Adopted Programming Languages . . . 14
3.2 Middleware . . . 15

4 Source Code Structure **16**
4.1 Backend Logic and Database Functions . . . 16

5 Testing **21**
5.1 Testing . . . 21
5.1.1 Geospatial Path Recognition . . . 21
5.1.2 Community Integrity and Data Protection . . . 21
5.1.3 Privacy and Workflow Consistency . . . 22
5.1.4 Ranking Algorithm Validation . . . 23

6 Installation Instructions **24**
6.1 Installation and Configuration . . . 24
6.1.1 Prerequisites . . . 24
6.1.2 Getting the Source Code . . . 24
6.1.3 Frontend Setup . . . 24
6.1.4 Environment and Database Connection . . . 25

6.1.5 Running the Application 25

7 Effort Spent **26**

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

List of Figures

List of Tables

5.1 Geospatial Similarity Test Cases 21
5.2 Community Integrity Test Cases 22
5.3 Privacy and Workflow Logic Test Cases 22
5.4 Ranking Algorithm Test Cases 23

7.1 Hours of work spent by each author on app implementation and document preparation . 26

Chapter 1

Introduction

1.1 Purpose

The purpose of this document (Implementation and Test Deliverable - ITD) is to describe the implementation of the Best Bike Paths (BBP) software system, specifying how the requirements defined in the RASD and the architecture designed in the DD have been translated into source code.

This document details the functionalities implemented in the final prototype, as well as the technologies and frameworks used for development. Furthermore, it describes the code structure, the strategies adopted to verify its correctness, and the necessary instructions for installation and execution.

In particular, this document highlights deviations from the original design, justifying the implementation choices made to adapt the project to time and technical constraints. It also describes the tools specifically developed to demonstrate the system's functionality.

1.2 Scope

The implemented system constitutes a functional prototype of the Best Bike Paths mobile application. The main objective of this implementation is to provide users (cyclists and tourists) with a tool to track their cycling activities, manage their route history, and visualize interactive maps.

Compared to the complete vision described in the RASD, the scope of this release focuses on the "Core" functionalities:

- **User Management:** Registration, authentication, and profile management.
- **Activity Tracking:** Real-time GPS tracking, calculation of basic statistics (distance, time, speed), and map visualization.
- **Content Management:** Route saving and history visualization.
- **Route Search:** Searching for routes visible to the user by entering departure and destination points.

Exclusions and Limitations In order to ensure the stability and quality of the application's main core, and in accordance with the Assignment requirements, some advanced features described in the design have been excluded from this version of the prototype:

- **Report Merge Logic:** The complex algorithm for the automatic unification of user reports has not been implemented.
- **Automatic Detection (Sensor-based):** Automatic pothole detection via accelerometer is not included.

Validation Support Features To allow for effective system verification in static test environments (e.g., classroom presentation) where physical GPS movement is not possible, the project scope has been extended to include a Route Simulator (Mocking System). This feature allows the application to generate fictitious GPS tracks and simulate user movement, enabling a complete demonstration of the "Start Activity → Tracking → End Activity → Data Analysis" flow without the need for physical displacement.

1.3 Definitions, Acronyms, and Abbreviations

1.3.1 Definitions

- **Backend-as-a-Service (BaaS)**: A cloud computing model adopted in this architecture where backend responsibilities (database, authentication, hosting) are outsourced to a managed provider (Supabase), allowing development to focus on frontend logic.
- **Cron Job**: A time-based job scheduler in Unix-like computer operating systems, used in this project to trigger periodic maintenance tasks (e.g., the decay of old feedbacks) on the server.
- **Edge Function**: A piece of serverless application logic (e.g., the path merging algorithm) that runs on the cloud provider's infrastructure, triggered by specific database events or HTTP requests.
- **Hybrid Application**: The chosen architectural approach where the mobile app is built using web technologies (React/TypeScript) but executes within a native container to access device hardware.
- **Kalman Filter**: An algorithm that uses a series of measurements observed over time (containing statistical noise) to produce estimates of unknown variables (e.g., precise GPS coordinates) that tend to be more accurate than those based on a single measurement alone.
- **Native Bridge**: The software mechanism (provided by Capacitor) that allows the web-based client code to communicate with the native operating system and access sensors like the Accelerometer and GPS in the background.
- **Object Storage**: A computer data storage architecture that manages data as distinct units (objects), used in this system to store large binary assets like user profile pictures.
- **PostGIS**: An open-source software extender for the PostgreSQL database, used in this project to store, index, and query complex geospatial data (geometries) efficiently.
- **Row Level Security (RLS)**: A security mechanism implemented directly in the database engine that restricts access to specific data rows based on the user's authentication token, eliminating the need for application-level security filters.
- **Thick Client**: An architectural style where the client application (running on the user's device) handles the bulk of data processing, validation, and temporary storage, contrasting with a Thin Client that relies heavily on the server.
- **Webhook**: A mechanism for an application to provide other applications with real-time information. In this system, database webhooks trigger Edge Functions automatically when data is inserted or updated.

1.3.2 Acronyms

- **API**: Application Programming Interface.
- **BaaS**: Backend-as-a-Service.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

- **CPU**: Central Processing Unit.
- **CRUD**: Create, Read, Update, Delete.
- **JSON**: JavaScript Object Notation.
- **JWT**: JSON Web Token (used for stateless authentication).
- **KPI**: Key Performance Indicator.
- **OS**: Operating System.
- **PWA**: Progressive Web Application.
- **RLS**: Row Level Security.
- **RPC**: Remote Procedure Call (used to invoke Edge Functions).
- **SDK**: Software Development Kit.
- **SPA**: Single Page Application.
- **SQL**: Structured Query Language.
- **WSS**: WebSocket Secure (used for real-time updates).

1.3.3 Abbreviations

- **DB**: DataBase.
- **UI**: User Interface.
- **VM**: Virtual Machine.

1.4 Revision History

Version 1.0: 01/02/2026

1.5 Reference Documents

- Requirement Analysis and Specification Document (RASD) v2.0
- Design Document (DD) v1.0
- Assignment RDD AY 2025-2026 - Software Engineering 2 Project Assignment, Politecnico di Milano.

1.6 Document Structure

This document is organized into the following chapters:

- **Chapter 1 Introduction:** Provides an overview of the document, defining the purpose and scope of the implementation. It explicitly states the boundaries of the prototype, highlighting the focus on core functionalities and the introduction of a "Demo Mode" for validation purposes in static environments.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

- **Chapter 2 Implemented Requirements and Functions:** Details the functional requirements that have been successfully implemented (e.g., User Management, Activity Tracking) and lists the functionalities excluded from this release (such as automatic sensor detection and merge logic), providing motivations based on project constraints. It also describes the additional "Mocking System" developed to support the demonstration.
- **Chapter 3 Adopted Development Frameworks and Technologies:** Describes the technology stack used to build the solution (including React, TypeScript, and Supabase), referencing the choices made in the Design Document and explaining the advantages of the adopted middleware and tools.
- **Chapter 4 Source Code Structure:** Explains the organization of the source code repository, detailing the folder hierarchy and the location of the main architectural components.
- **Chapter 5 Testing:** Outlines the testing strategies employed to ensure software quality, describing the procedures used for system testing and the validation of the core user flows through the implemented prototype.
- **Chapter 6 Installation Instructions:** Provides step-by-step instructions for setting up the development environment, installing necessary dependencies, and executing the software to reproduce the implemented features.
- **Chapter 7 Effort Spent:** Summarizes the time effort dedicated by each team member to the development of the code and the writing of this document.

1.7 Use of Generative AI Tools

In compliance with the course guidelines regarding academic integrity and content responsibility, we declare that we have used Generative Artificial Intelligence (GenAI) tools to support the drafting of this document (Design Document - DD).

We emphasize that the entire content of this document is the result of our original reasoning; AI was not used to generate architectural decisions or core logic from scratch, but exclusively as a tool for linguistic support, formal verification, and visual prototyping. The details of its usage are reported below:

1. **Tools Used**
   - **Textual Support:** ChatGPT (model GPT-3.5/4o mini), Gemini Advanced (Gemini 3).
   - **Visual Generation (UI Mockups):** Lovable.
2. **Provided Inputs (Prompts and Context)** The inputs provided to the system were exclusively textual and based on preliminary work carried out by the group. Specifically:
   - **For linguistic revision:** We input paragraphs or entire sections written by us in Italian/English, with the request to improve grammatical form, academic vocabulary, and text fluidity, without altering the technical meaning.

     Example A - For stylistic improvement: *"Act as a professional Technical Writer reviewing a Software Design Document. I will provide you with a raw text describing the 'Path Merging Algorithm'. Please rewrite it to improve grammatical correctness and use formal academic English suitable for a system architecture description. Do NOT add new information or change the logical flow. Just polish the language."*

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

- **For completeness verification:** We provided lists of component interfaces or sequence diagram steps developed by the group, asking the model to flag potential logical ambiguities or missing edge cases.

  Example B - For completeness verification: *"Act as a Senior Software Engineer. Below is the list of APIs we defined for the 'Sensor Controller'. Analyze them for consistency with the REST standard. Specifically, identify if there are any missing response codes or parameters that a standard Design Document should include."*

- **For UI Prototyping (via Lovable):** We provided detailed descriptions of the application screens, including layout structure, color palettes, and required UI elements, to generate high-fidelity visual mockups for Chapter 3.

  Example C - For UI Generation: *"Create a mobile screen for a cycling app in a modern style. It must include a map view showing a route, a bottom sheet showing real-time speed (25 km/h) and distance (12.4 km), and a large red circular button to 'Report Hazard'."*

3. **Obtained Outputs**

   - Suggestions for sentence reformulation to make the text more formal and concise.
   - Checklists to verify consistency between different diagrams and textual descriptions.
   - Graphical assets and UI Mockups representing the application interface, which were subsequently refined to match the UX flow.

4. **Verification, Refinement, and Integration** Every output generated by the AI was subjected to a manual critical review by the group members.

   - We did not passively accept the proposed reformulations; instead, we selected and adapted them to ensure they exactly reflected our project specifications.
   - The UI mockups generated by Lovable were manually reviewed and used as a basis to define the final interface design, ensuring consistency with the functional requirements.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Chapter 2

Implemented Requirements and Functions

2.1 Implemented Requirements

This section outlines the requirements satisfied by the current implementation, details those that were excluded, and describes the additional functionalities introduced.

2.1.1 Implemented Functional Requirements

- **User Management (R1, R2, R4):** Users can choose to register and log in or access the application as a guest.
- **Activity Recording (R5, R6, R7, R10, R11):** Users can start, pause, and stop the recording of their activity. The system calculates basic statistics (such as speed) and tracks the route via GPS in real-time.
- **Route Creation (R12, R13 [partial], R14, R15, R17, R18, R19):** Upon completion of an activity, detected obstacles are displayed (note: these are currently simulated/fictitious) and can be deleted by the user. A new route is created from the completed activity, subject to a matching process with existing routes to prevent duplication. Once created, routes are saved to "My Paths".
- **Manual Creation (R20, R21, R22):** Users can manually create a route by drawing it on the map, with the ability to report obstacles and define the route status. Upon completion, the route is saved to "My Paths".
- **Search and Visualization (R23, R25):** Users can search for routes by specifying a starting point and a destination. Results are sorted in descending order based on the score assigned by the search algorithm.
- **Route Management (R28, R29):** Users can publish their own routes. Once a route is published, it is considered immutable and can no longer be deleted or updated.

2.1.2 Excluded or Partially Implemented Functionalities

Functionalities excluded in accordance with the project assignment specifications:

- **Automatic Obstacle Detection (R8, R9):** Detection via accelerometer/gyroscope was excluded. Validation is partial as data is not automatically detected but simulated. Furthermore, velocity checks to confirm cycling patterns are disabled, primarily because the mocking function generates activities faster than actual cycling speeds.
- **Merge Logic (R16, R26, R27):** The complex logic for merging reports to update the status of public routes has been excluded, along with the general management of user reports.

Functionalities excluded to focus on the project's core features:

- **Route Search Filter (R24):** Routes are filtered only within a fixed radius from the origin and destination points, excluding the ability for the user to customize filter parameters.
- **Credential Management (R3):** The modification of credentials was excluded as registration and authentication are handled exclusively via Google OAuth.

2.1.3 Additional Functionalities

- **Fictitious Route Generation (Mocking):** Fictitious Route Generation (Mocking): Implementation of a simulated route generator that creates plausible GPS tracks. Crucially, the system randomly injects obstacles (hazards) along the path during the generation process. The primary purpose is to enable the demonstration of the "Activity Recording", "Statistics Visualization", "Map Interaction", "Hazard Detection", and "Activity Completion" flows without physical movement. Additionally, this feature facilitates the development and testing of these workflows, ensuring that the system's ability to identify and report environmental dangers can be verified in a controlled environment.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Chapter 3

Adopted Development Frameworks and Technologies

3.1 Adopted Programming Languages

For the development of the **BestBikePaths** platform, the architecture of choice is a **Progressive Web App (PWA)**. This technology bridges the gap between traditional web applications and native mobile software. By utilizing a web app manifest, the platform can be installed directly onto a user's device, providing a native-like cross-platform experience from a single codebase. This approach significantly optimizes the development lifecycle, testing procedures, and long-term maintenance.

While PWAs can occasionally encounter limitations regarding offline data persistence and deep hardware integration, current technological standards provide robust solutions to these challenges:

- **Connection Resilience:** Through the implementation of **Service Workers**, the application can intercept network requests and manage advanced caching strategies, ensuring that core functionalities remain available even in offline environments.
- **Hardware Integration:** Regarding access to system sensors (such as the accelerometer, camera, or microphone), frameworks like **Capacitor** offer a solution by providing a standardized interface between web code and the native APIs of iOS and Android, effectively addressing interoperability constraints.

TypeScript: Advantages and Disadvantages

The application was developed using **TypeScript**, a superset of JavaScript that introduces static typing. Its adoption within the project is justified by the following technical analysis:

Advantages

- **Static Typing and Reliability:** TypeScript allows for the detection of type-related errors during the compilation phase rather than at runtime. This leads to more stable code and a significant reduction in production bugs.
- **Maintainability and Scalability:** The use of explicit interfaces and custom types ensures the codebase is self-documenting. This facilitates collaborative development and simplifies the process of scaling the application as new features are added.
- **Enhanced Developer Tooling:** TypeScript provides superior IDE support, including advanced autocompletion, precise navigation, and safer refactoring, which collectively increase development velocity.

Disadvantages

- **Transpilation Requirement:** TypeScript cannot be executed natively by web browsers. It requires a compilation (transpilation) step into JavaScript, which adds complexity to the automated build pipeline.
- **Learning Curve:** Proper implementation of strict typing and advanced architectural patterns requires a higher level of technical expertise compared to standard JavaScript.
- **Initial Verbosity:** The requirement to explicitly define data structures results in more boilerplate code during the early stages of development, which can slightly increase the time required for rapid prototyping.

3.2 Middleware

Serverless Logic and Infrastructure: Supabase Edge Functions

To handle server-side business logic and third-party integrations, the project utilizes **Supabase Edge Functions**. These are serverless TypeScript functions executed across a distributed Edge Network—a global infrastructure of nodes (Points of Presence) designed to process data at the "edge" of the internet, physically closer to the end-user than traditional centralized cloud servers.

The adoption of this architecture provides several strategic advantages and some trade-offs:

Advantages

- **Minimal Latency:** By executing logic on the edge node nearest to the user, the application significantly reduces data travel time, ensuring a highly responsive experience for mobile users in motion.
- **Enhanced Security:** Edge Functions provide a secure environment to manage private API keys and sensitive validation logic, keeping them isolated from the client-side environment.
- **Automated Scalability:** The serverless nature of the infrastructure allows it to scale dynamically with traffic, removing the need for manual server provisioning or maintenance.
- **Native Ecosystem Integration:** Full compatibility with Supabase Auth and PostgreSQL enables the seamless implementation of complex workflows, such as secure data processing and automated notifications.

Disadvantages

- **Cold Starts:** Infrequently used functions may encounter a minor delay during the initial invocation as the execution environment warms up.
- **Resource Limits:** Compared to traditional dedicated servers, Edge Functions operate within stricter memory and execution time constraints.
- **Distributed Debugging:** Testing and monitoring logic across a global network can be more complex than managing a centralized, monolithic backend.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Chapter 4

Source Code Structure

The codebase is engineered to promote modularity and scalability, ensuring that individual components can be updated or reused without compromising the integrity of the entire system. A strict separation of concerns has been enforced by organizing the project into distinct directories, effectively decoupling UI elements, business logic, and external service integrations. All core PWA source code is contained within the `src` directory, structured as follows:

Components: This directory contains both bespoke project components and those derived from the **shadcn/ui** library. The `ui` subfolder is reserved for the foundational shadcn library components, while the `app` subfolder and the root of the directory house custom-built elements.

Hooks: This folder contains custom React Hooks, which encapsulate reusable logic to maintain clean components. They are specifically utilized for **Device Detection** (adapting the UI for mobile vs. desktop) and **Toast Notifications** (managing system feedback).

Pages: This directory follows a file-based routing logic, containing the main views of the application. The `app` subfolder organizes core views such as the **Dashboard**, **ExplorePaths**, and **RecordTrip**, ensuring a clear mapping between URL paths and UI controllers.

Stores: This directory houses global state management logic (e.g., via Zustand). Key files include `useAuthStore.ts` for authentication states and `useDebugStore.ts` for development-time performance monitoring.

Lib: Dedicated to utility functions and service configurations. It includes `utils.ts` for shared helpers and specialized modules like `weatherService.ts`, which abstracts the logic for processing external meteorological data.

4.1 Backend Logic and Database Functions

The backend logic is implemented using Supabase, leveraging PostgreSQL for data storage and spatial operations (PostGIS), and Edge Functions for complex computational logic. Below is a description of the key functions and triggers implemented.

Edge Functions

- **func_path_search:** This TypeScript-based Edge Function acts as the main entry point for the route search algorithm. It handles the API request containing origin and destination coordinates and invokes the database RPC `search_best_paths_rpc` to retrieve candidate paths. Crucially, it implements a ranking algorithm that sorts results based on a composite score derived from:

- **Geometric Quality:** Derived from the spatial match score (exponential decay).
- **Data Freshness:** Penalizing paths that haven't been updated recently (time-decay factor).
- **Path Status:** Weighting paths classified as "optimal" higher than those with "sufficient" or "bad" conditions.

Code:

```
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";

const corsHeaders = {
"Access-Control-Allow-Origin": "*",
"Access-Control-Allow-Headers": "authorization, x-client-info, apikey,
content-type",
};

Deno.serve(async (req) => {
if (req.method === "OPTIONS") {
    return new Response("ok", { headers: corsHeaders });
}

try {
    const supabaseClient = createClient(
    Deno.env.get("SUPABASE_URL") ?? "",
    Deno.env.get("SUPABASE_ANON_KEY") ?? "",
    {
        global: {
        headers: { Authorization: req.headers.get("Authorization")! },
        },
    }
    );

    const { origin_lat, origin_lon, dest_lat, dest_lon } = await req.
json();

    const { data: candidates, error } = await supabaseClient.rpc(
    "search_best_paths_rpc",
    { origin_lat, origin_lon, dest_lat, dest_lon, buffer_meters: 20000}
    );

    if (error) throw error;

    if (!candidates || candidates.length === 0) {
    return new Response(JSON.stringify([]), {
        headers: { ...corsHeaders, "Content-Type": "application/json" }
    });
    }

    const rankedPaths = candidates.map((path: any) => {
    // Geometric Quality: exp(-0.001 * x)
    let geo_match_score = 0;
    if(path.geo_match_score >= path.geo_match_score_rev){
        geo_match_score = path.geo_match_score_rev;
    }else{
        geo_match_score = path.geo_match_score;
    }
    const geometricQuality = Math.exp(-0.001 * geo_match_score);

    // Days since last status update
    const daysSinceUpdate = (new Date().getTime() - new Date(path.
last_update).getTime()) / (1000 * 3600 * 24);
    // Every 30 days the score lost 10% of his value
```

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

```
        const freshnessFactor = Math.exp(-0.003 * daysSinceUpdate);

        let ratingStatus = 0;
        switch (path.status) {
            case "optimal": ratingStatus = 1; break;
            case "good": ratingStatus = 0.8; break;
            case "sufficient": ratingStatus = 0.6; break;
            default: ratingStatus = 0.3;
        }

        const finalScore = ((geometricQuality * 0.4) + (ratingStatus *
    freshnessFactor * 0.6))*100;

        return {
            ...path,
            score: finalScore,
        };
        });

        rankedPaths.sort((a: any, b: any) => b.score - a.score);

        return new Response(JSON.stringify(rankedPaths.slice(0, 10)), {
        headers: { ...corsHeaders, "Content-Type": "application/json" },
        status: 200,
        });

    } catch (error) {
        return new Response(JSON.stringify({ error: error.message }), {
        headers: { ...corsHeaders, "Content-Type": "application/json" },
        status: 400,
        });
    }
    });
```

Database Functions (RPCs)

- **check_path_similarity:** A PostGIS spatial function designed to prevent the creation of duplicate paths. Before saving a new route, this function compares the geometry of the new activity against existing paths using the **Fréchet distance** and length tolerance. If a matching path is found (within the specified tolerance meters), the system associates the activity with the existing path instead of creating a new one.
  Code:

```
    DECLARE
        new_geom GEOMETRY;
        new_len FLOAT;
    BEGIN
        new_geom := ST_SetSRID(ST_GeomFromText(new_geom_wkt), 4326);
        new_len := ST_Length(new_geom::geography);

        RETURN QUERY
        SELECT
            p.id,
            p.name,
            p.description,
            p.id_owner,
            p.is_public,
            ST_FrechetDistance(
                ST_Transform(p.geom::geometry, 3857),
```

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

```
            ST_Transform(new_geom, 3857)
        ) as similarity_score
    FROM
        paths p
    WHERE
        p.geom IS NOT NULL
        AND (p.is_public = true OR p.id_owner = auth.uid())
        AND ABS(ST_Length(p.geom::geography) - new_len) < (new_len *
length_tolerance_pct)
        AND ST_DWithin(p.geom::geography, new_geom::geography,
tolerance_meters * 2)
        AND ST_FrechetDistance(ST_Transform(p.geom::geometry, 3857),
ST_Transform(new_geom, 3857)) < tolerance_meters
    ORDER BY similarity_score ASC
    LIMIT 5;
END;
```

- **get_path_community_usage:** A utility function that calculates the popularity of a specific path. It counts the number of distinct users (excluding the current requester) who have recorded activities on that path, providing a metric for community usage.
  Code:

```
DECLARE
usage_count int;
BEGIN
IF p_path_id IS NULL THEN
    RETURN 0;
END IF;

SELECT count(distinct user_id)
INTO usage_count
FROM activities
WHERE path_id = p_path_id
AND user_id != auth.uid();
END;
```

- **search_best_paths_rpc:** The core spatial query invoked by the Edge Function. It utilizes PostGIS functions (such as `ST_DWithin` and `ST_Distance`) to efficiently query the `paths` table. It identifies all public or user-owned paths that spatially connect the requested origin and destination points within a 20km buffer, calculating the geometric match score for both forward and reverse directions.
  Code:

```
DECLARE
    origin_pt geography;
    dest_pt geography;
BEGIN
    origin_pt := ST_SetSRID(ST_MakePoint(origin_lon, origin_lat), 4326)
::geography;
    dest_pt := ST_SetSRID(ST_MakePoint(dest_lon, dest_lat), 4326)::
geography;

    RETURN QUERY
    SELECT
    p.id, p.name, p.status, p.last_update,
    (
        ST_Distance(ST_StartPoint(p.geom)::geography, origin_pt) +
        ST_Distance(ST_EndPoint(p.geom)::geography, dest_pt)
```

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

```
        )::float8 as geo_match_score,
        (
            ST_Distance(ST_StartPoint(p.geom)::geography, dest_pt) +
            ST_Distance(ST_EndPoint(p.geom)::geography, origin_pt)
        )::float8 as geo_match_score_rev
        FROM public.paths p
        WHERE
        (
            p.is_public IS TRUE OR p.id_owner = auth.uid()
        )
        AND
        (
            (
            ST_DWithin(ST_StartPoint(p.geom)::geography, origin_pt,
    buffer_meters)
            AND
            ST_DWithin(ST_EndPoint(p.geom)::geography, dest_pt,
    buffer_meters)
            )
            OR
            (
            ST_DWithin(ST_StartPoint(p.geom)::geography, dest_pt,
    buffer_meters)
            AND
            ST_DWithin(ST_EndPoint(p.geom)::geography, origin_pt,
    buffer_meters)
            )
        );
    END;
```

Database Triggers

- **before_insert_set_last_update:** Ensures data consistency by automatically initializing the `last_update` timestamp to the record's creation time (`created_at`) upon insertion.
  Code:

```
    BEGIN
    NEW.last_update := NEW.created_at;
    RETURN NEW;
    END;
```

- **set_last_update_after_status_update:** Monitors changes to the `status` column of a path. Whenever the status is modified (e.g., from "good" to "bad"), this trigger automatically refreshes the `last_update` timestamp to `now()`, ensuring the "Data Freshness" score in the search algorithm remains accurate.
  Code:

```
    BEGIN
    IF OLD.status IS DISTINCT FROM NEW.status THEN
        UPDATE public.paths
        SET last_update = now()
        WHERE id = NEW.id;
    END IF;
    RETURN NEW;
    END;
```

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Chapter 5

Testing

5.1 Testing

The implementation has undergone a rigorous testing phase focusing on the geospatial algorithms, data integrity constraints, user permission logic (Row Level Security), and privacy-aware workflows. The following subsections describe the main validation scenarios designed to ensure correctness, usability, and protection of community data.

5.1.1 Geospatial Path Recognition

This module verifies the system's ability to identify whether a newly recorded activity matches an existing path in the database using the PostGIS-powered `check_path_similarity` RPC function.

ID	Test Scenario	Expected Outcome	Result
GEO-01	**Perfect Match:** User records an activity that strictly follows the geometry of an existing public path within the 50m tolerance buffer.	The system identifies the existing path ID. The UI displays a "Match Found" dialog prompting the user to link the activity to the existing path instead of creating a new one.	**Pass**
GEO-02	**Partial Match / Deviation:** User records a track that overlaps with an existing path for 40% of the length but deviates significantly for the rest.	The system returns no match (null). The UI prompts the user to create a new path with a new name and description.	**Pass**
GEO-03	**Reverse Direction:** User traverses an existing path starting from the end point to the start point.	The algorithm recognizes the path regardless of the traversal direction.	**Pass**

Table 5.1: Geospatial Similarity Test Cases

5.1.2 Community Integrity and Data Protection

These tests validate the logic governed by the `get_path_community_usage` function and RLS policies, ensuring that public resources used by the community cannot be deleted or modified in ways that would compromise data integrity.

ID	Test Scenario	Expected Outcome	Result
SEC-01	**Safe Deletion:** An owner attempts to delete a path with 0 associated activities from other users.	The deletion proceeds successfully. The path and the owner's personal activities are removed (cascade delete).	**Pass**
SEC-02	**Protected Deletion:** An owner attempts to delete a public path that has been used by at least one other rider.	The database rejects the operation. The frontend displays a warning: "Cannot delete: Path is actively used by X riders".	**Pass**
SEC-03	**Privatization Attempt:** An owner switches a path from public to private while it has community activities.	The operation is blocked to preserve visibility and consistency of linked activities.	**Pass**

Table 5.2: Community Integrity Test Cases

5.1.3 Privacy and Workflow Consistency

This subsection validates the advanced workflow logic designed to prevent data duplication and respect user ownership priority during path matching and publication.

ID	Test Scenario	Expected Outcome	Result
WRK-01	**Ownership Priority:** User A has a private path P_{priv}. An identical public path P_{pub} also exists. User A records an activity on this route.	The system detects both paths but prioritizes ownership. The UI automatically links the activity to P_{priv} ("Welcome Back") without prompting to link to the public version.	**Pass**
WRK-02	**Tracking Opt-Out:** User records an activity on a public path but chooses not to appear on the leaderboard (unchecks "Link Activity").	The system creates a new **Private Copy** of the path for the user. The activity is linked to this new private entity, ensuring isolation from public stats.	**Pass**
WRK-03	**Publication Conflict:** User attempts to toggle a private path to "Public", but an identical public path already exists in the system.	The system blocks the direct status change to prevent duplicates. A dialog offers a choice: Merge into the existing path or cancel the operation.	**Pass**
WRK-04	**Merge Resolution:** User accepts the "Merge" option during a publication conflict.	The private path is deleted. All associated activities are transferred to the existing public path. The user receives a success confirmation.	**Pass**
WRK-05	**Passive Cancellation:** User chooses "Keep Private" during a publication conflict.	The publication process is aborted. The path remains private and no changes are made to the database (zero-side-effect action).	**Pass**

Table 5.3: Privacy and Workflow Logic Test Cases

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

5.1.4 Ranking Algorithm Validation

This subsection verifies the multi-variable scoring formula implemented in the `func_path_search` Edge Function. The tests isolate specific factors (Geometry, Status, Freshness) to ensure each component correctly influences the final sorting order.

ID	Test Scenario	Expected Outcome	Result
RNK-01	**Direction Invariance:** Perform a search $A \rightarrow B$ and then immediately $B \rightarrow A$.	The system returns identical scores for the same path in both directions, confirming that `geo_match_score` and `geo_match_score_rev` are handled symmetrically.	**Pass**
RNK-02	**Status Weighting:** Two paths have identical geometry and update timestamps. Path P_{opt} is "Optimal", Path P_{suff} is "Sufficient".	P_{opt} appears higher in the list. The score difference reflects the status multiplier (1.0 vs 0.6), proving the quality attribute is active.	**Pass**
RNK-03	**Data Freshness (Time Decay):** Two identical paths with "Good" status exist. Path P_{new} was updated today, Path P_{old} was updated 1 year ago.	P_{new} ranks significantly higher than P_{old}. This validates the `freshnessFactor` (exponential decay), ensuring users are directed to recently verified routes.	**Pass**
RNK-04	**Geometric Proximity:** User searches for a route from point O to D. Path P_{near} starts/ends exactly at O/D. Path P_{far} starts/ends 500m away.	P_{near} obtains a higher geometric score ($e^{-0.001 \times distance}$) compared to P_{far}, confirming that the algorithm prioritizes paths that minimize the distance to the user's requested endpoints.	**Pass**
RNK-05	**Result Limiting:** Perform a broad search in a dense area with 50+ potential paths.	The API returns exactly the top 10 results sorted by score descending, validating the performance limit logic.	**Pass**

Table 5.4: Ranking Algorithm Test Cases

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Chapter 6

Installation Instructions

6.1 Installation and Configuration

This chapter provides a comprehensive guide to setting up the development environment, configuring the backend services, and deploying the application locally. The source code is version-controlled and hosted on GitHub.

6.1.1 Prerequisites

Before proceeding with the installation, ensure that the following software is installed on the local machine:

- **Node.js** (Version 18.0 or higher): Required for the React runtime environment.
- **npm** (Node Package Manager) or **yarn**: Required for dependency management.
- **Git**: Required for cloning the repository.

6.1.2 Getting the Source Code

The complete source code for the *Best Bike Path* application is hosted on GitHub. To begin, clone the main repository to your local workspace and navigate to the web application directory:

```
$ git clone https://github.com/Alessio01103/PesaventoSpezzapriaSuzani.git
$ cd PesaventoSpezzapriaSuzani/IT/ITD/BestBikePaths
```

All subsequent commands (installation, startup) must be executed from within this `BestBikePaths` directory.

6.1.3 Frontend Setup

Installing Dependencies

Once inside the project directory, install the required node modules defined in the `package.json` file. This includes the React framework, UI components (shadcn/ui), mapping libraries (Leaflet), and Supabase client SDKs.

```
$ npm install
```

6.1.4 Environment and Database Connection

The application relies on environment variables to communicate with the Supabase backend (PostgreSQL database and Authentication services).

Pre-configured Credentials

To facilitate the testing and evaluation process, the repository **already includes** the `.env` file populated with the necessary API credentials:

```
VITE_SUPABASE_URL
VITE_SUPABASE_ANON_KEY
```

Therefore, **no manual configuration is required** to connect to the database. The application is pointed to a shared testing instance of Supabase populated with mock data for demonstration purposes.

> **Note on Security Best Practices:**
> We explicitly acknowledge that committing sensitive credentials (such as API Keys) to a version control system is considered a security anti-pattern in production environments. However, for the specific scope of this academic project, we consciously chose to include the configuration file to ensure immediate reproducibility and eliminate setup friction for the evaluation committee. The exposed keys grant access only to a sandbox environment containing non-sensitive test data.

6.1.5 Running the Application

Since the database connection is pre-configured, you can immediately start the local development server:

```
$ npm run dev
```

The application will be accessible at `http://localhost:8080`.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Chapter 7

Effort Spent

Task	Date	Alessio P.	Alberto S.	Luca S.
App Implementation	10/01/2026	3h	2h	2h
	12/01/2026	2h	3h	2h
	15/01/2026	2h	2h	3h
	18/01/2026	1.5h	2h	2h
	20/01/2026	2h	3h	1.5h
	23/01/2026	1h	2h	2h
	27/01/2026	2h	2h	2h
	29/01/2026	3h	3h	2h
	31/01/2026	4h	6h	4h
	Total	**20.5h**	**25h**	**20.5h**
Document Preparation	22/01/2026	2h	1.5h	1h
	24/01/2026	2h	2h	1.5h
	26/01/2026	1.5h	0.5h	2h
	28/01/2026	2h	0.0h	1h
	01/02/2026	3h	3h	3h
	Total	**10.5h**	**7h**	**8.5h**
TOTAL		**31h**	**32h**	**29h**

Table 7.1: Hours of work spent by each author on app implementation and document preparation